3D Gaussian Splatting for Real-Time Radiance Field Rendering

Bernhard Kerbl et al. · SIGGRAPH 2023

MOAI Seminar | Presenter 김태하

Introduction

Novel View Synthesis

- Input: multiple photographs of a static scene
- Output: photorealistic images from unseen camera poses
- Requires geometry, appearance, and visibility reasoning

Why Is Real-Time Rendering Difficult?

- High-quality radiance fields require expensive rendering
- Ray marching evaluates many samples along every ray
- Speed, training time, and visual quality have conflicted

Explicit vs. Implicit Representations

- Explicit: points / meshes enable GPU-friendly rasterization
- Implicit: continuous radiance fields support optimization
- 3DGS combines explicit primitives with differentiable rendering

NeRF: The Starting Point

- NeRF models color and density along camera rays
- Volume rendering accumulates transmittance-weighted colors
- Dense sampling and network evaluation are expensive

Volume Rendering Equation

- $C = \Sigma_i T_i \alpha_i c_i$
- $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$
- $T_i = \Pi_{j<i} (1 - \alpha_j)$

Point-Based Alpha Blending

- $C = \Sigma_i c_i \alpha_i \Pi_{j<i}(1 - \alpha_j)$
- Same compositing form as discrete volume rendering
- Different implementation: rasterize splats, not sample rays

Prior Methods and Their Trade-offs

- Mip-NeRF360: strong image quality, slow training/rendering
- InstantNGP: fast optimization using hash grids
- Plenoxels: sparse voxel grid without an MLP

Three Main Contributions

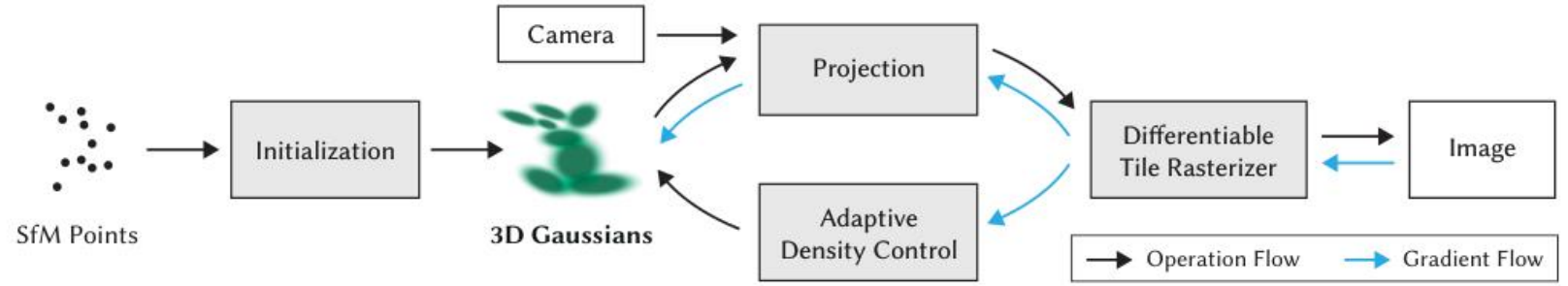


Fig. 2. Optimization starts with the sparse SfM point cloud and creates a set of 3D Gaussians. We then optimize and adaptively control the density of this set of Gaussians. During optimization we use our fast tile-based renderer, allowing competitive training times compared to SOTA fast radiance field methods. Once trained, our renderer allows real-time navigation for a wide variety of scenes.



We use $\lambda = 0.2$ in all our tests. We provide details of the learning schedule and other elements in Sec. 7.1.

5.2 Adaptive Control of Gaussians

We start with the initial set of sparse points from SfM and then apply

- Anisotropic 3D Gaussian scene representation
- Interleaved optimization and adaptive density control
- Fast visibility-aware differentiable tile rasterizer

Method: Representation

Pipeline Overview

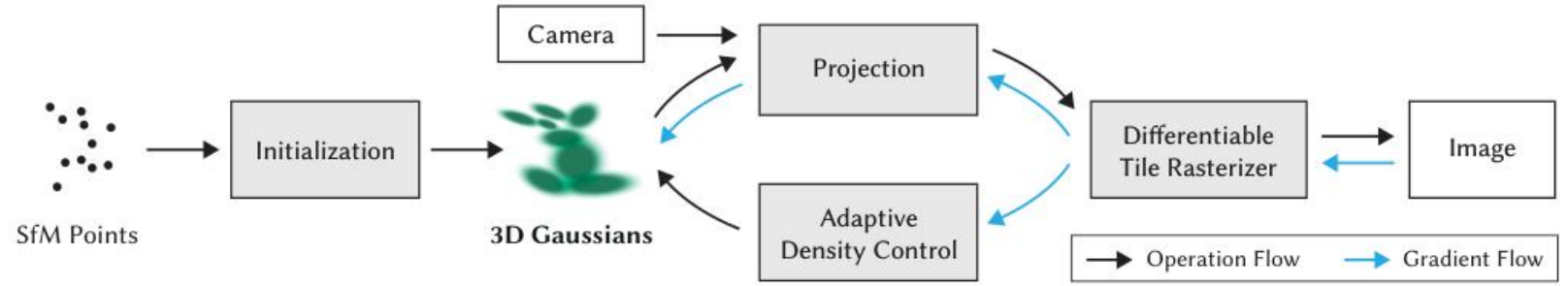


Fig. 2. Optimization starts with the sparse SfM point cloud and creates a set of 3D Gaussians. We then optimize and adaptively control the density of this set of Gaussians. During optimization we use our fast tile-based renderer, allowing competitive training times compared to SOTA fast radiance field methods. Once trained, our renderer allows real-time navigation for a wide variety of scenes.



We use $\lambda = 0.2$ in all our tests. We provide details of the learning schedule and other elements in Sec. 7.1.

5.2 Adaptive Control of Gaussians

We start with the initial set of sparse points from SfM and then apply

- SfM estimates camera poses and sparse 3D points
- Initialize Gaussians, render training views, compare images
- Optimize Gaussian attributes and adapt their density

Input: Structure-from-Motion

- Calibrated images + sparse point cloud
- No dense MVS geometry or point normals required
- Each sparse point seeds a Gaussian

What Is a 3D Gaussian?

- Mean μ determines 3D position
- Covariance Σ determines size and orientation
- Opacity α and spherical harmonics encode appearance

Gaussian Density in 3D

- $G(x) = \exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu))$
- Level sets form ellipsoids in 3D space
- Continuous and differentiable with respect to parameters

Why Anisotropic Gaussians?

- Isotropic: same spread in every direction
- Anisotropic: different scales along principal axes
- Thin surfaces and elongated structures use fewer primitives

Covariance Parameterization

- $\Sigma = R\,S\,S^T\,R^T$
- S: diagonal scaling matrix; R: rotation matrix
- Optimize scale vector s and quaternion q separately

Why Not Optimize Σ Directly?

- Covariance must remain positive semidefinite
- Unconstrained gradient steps can invalidate Σ
- Scale/rotation parameterization preserves a valid shape

Projecting a Gaussian into 2D

- Transform 3D covariance into camera coordinates
- $\Sigma' = J\,W\,\Sigma\,W^T\,J^T$
- Use the 2×2 image-plane covariance for splatting

View-Dependent Color with SH

- Spherical harmonics encode directional appearance
- Per-Gaussian SH coefficients are optimized
- Supports color changes as the camera moves

From 3D Ellipsoids to 2D Splats

- Project Gaussian mean and covariance
- Evaluate pixel coverage using the 2D Gaussian
- Composite visible splats in approximate depth order

Method: Optimization

Optimized Parameters

- Position p and scale/rotation (covariance)
- Opacity α and spherical harmonics coefficients
- All updated using image reconstruction gradients

Reconstruction Loss

- $L = (1-\lambda)L_1 + \lambda L_D\text{-}SSIM$
- $\lambda = 0.2$ in the paper
- Combines pixel accuracy with structural similarity

Training Stabilization

- Sigmoid constrains opacity to [0,1)
- Exponential activation keeps scales positive
- Start with lower resolution; gradually increase it

Why Adaptive Density Control?

- Sparse SfM initialization misses geometry
- Some Gaussians become too large or redundant
- Optimization must create, move, and remove primitives

Densification: Clone vs. Split

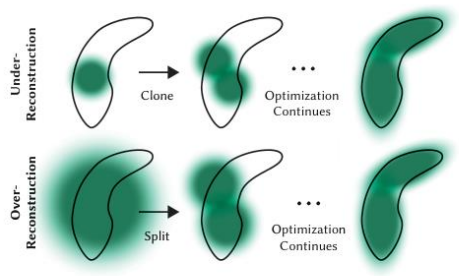


Fig. 4. Our adaptive Gaussian densification scheme. *Top row (under-reconstruction)*: When small-scale geometry (black outline) is insufficiently covered, we clone the respective Gaussian. *Bottom row (over-reconstruction)*: If small-scale geometry is represented by one large splat, we split it in two.

iterations. The optimization then increases the α for the Gaussians where this is needed while allowing our culling approach to remove Gaussians with α less than ϵ_α as described above. Gaussians may shrink or grow and considerably overlap with others, but we periodically remove Gaussians that are very large in worldspace and those that have a big footprint in viewspace. This strategy results in overall good control over the total number of Gaussians. The Gaussians in our model remain primitives in Euclidean space at all

- High view-space position gradients identify poorly fit areas
- Small Gaussians: clone to cover missing structure
- Large Gaussians: split into smaller primitives

Density Control Schedule

- Densify every 100 iterations after warm-up
- Remove nearly transparent Gaussians
- Reset opacity periodically; prune overly large splats

Method: Rasterization

Tile-Based Rasterization

- Partition the screen into 16×16 pixel tiles
- Cull Gaussians outside the view and relevant tiles
- Duplicate instances for all overlapping tiles

Depth Sorting on the GPU

- Create a key from tile ID and view-space depth
- Sort instances using fast GPU radix sort
- Generate contiguous depth-ordered ranges per tile

Forward Rendering

- One GPU thread block per tile
- Load Gaussian batches into shared memory
- Front-to-back α-blending; stop at opacity saturation

Approximate Visibility Ordering

- Tile-level sorting avoids per-pixel sorting
- Approximation may occur when splats overlap in depth
- Authors report negligible visible artifacts after convergence

Differentiable Backward Pass

- Revisit sorted tile lists in back-to-front order
- Reuse saved accumulated opacity information
- No fixed limit on the number of contributing splats

Why Is the Renderer Fast?

- Explicit projected primitives avoid ray marching
- Tile batching and shared memory improve GPU utilization
- Early termination avoids unnecessary blending

Experiments & Evaluation

Experimental Setup

- Real-world: Mip-NeRF360, Tanks&Temples, Deep Blending
- Metrics: PSNR ↑, SSIM ↑, LPIPS ↓
- Also compare training time, FPS, and model memory

Quantitative Comparison

Table 1. Quantitative evaluation of our method compared to previous work, computed over three datasets. Results marked with dagger † have been directly adopted from the original paper, all others were obtained in our own experiments.

Dataset		Mip-NeRF360						Tanks&Temples						Deep Blending					
Method	Metric	*SSIM*↑	*PSNR*↑	*LPIPS*↓	Train	FPS	Mem	*SSIM*↑	*PSNR*↑	*LPIPS*↓	Train	FPS	Mem	*SSIM*↑	*PSNR*↑	*LPIPS*↓	Train	FPS	Mem
Plenoxels		0.626	23.08	0.463	25m49s	6.79	2.1GB	0.719	21.08	0.379	25m5s	13.0	2.3GB	0.795	23.06	0.510	27m49s	11.2	2.7GB
INGP-Base		0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big		0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360		0.792†	27.69†	0.237†	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K		0.770	25.60	0.279	6m25s	160	523MB	0.767	21.20	0.280	6m55s	197	270MB	0.875	27.78	0.317	4m35s	172	386MB
Ours-30K		0.815	27.21	0.214	41m33s	134	734MB	0.841	23.14	0.183	26m54s	154	411MB	0.903	29.41	0.243	36m2s	137	676MB



is observed by photos taken in the entire hemisphere around it, the optimization works well. However, if the capture has angular regions missing (e.g., when capturing the corner of a scene, or performing an "inside-out" [Hedman et al. 2016] capture) completely incorrect values for the zero-order component of the SH (i.e., the base or

- Ours-30K reaches 134 FPS on Mip-NeRF360
- Mip-NeRF360 baseline: 0.06 FPS in Table 1
- 3DGS uses substantially more model memory

Qualitative Rendering Results

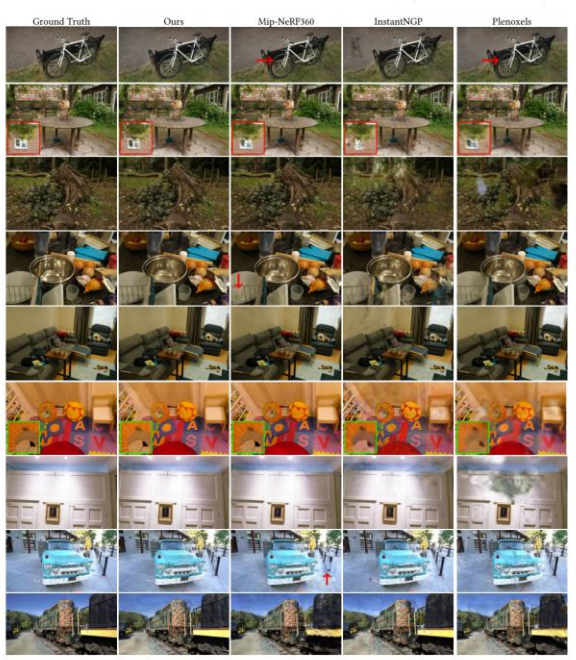


Fig. 5. We show comparisons of ours to previous methods and the corresponding ground truth images from held-out test views. The scenes are, from the top down: Bicycle, Garden, Stump, Counter and Room from the Mip-NeRF360 dataset; Playroom, DrJohnson from the Deep Blending dataset [Hedman et al. 2018] and Truck and Train from Tanks&Temples. Non-obvious differences in quality highlighted by arrows/insets.

- Compare ground truth against competing methods
- Fine structures and complex backgrounds are reconstructed
- Image quality remains competitive with high-quality NeRF

Training Progress: 7K vs. 30K



Fig. 6. For some scenes (above) we can see that even at 7K iterations (~5min for this scene), our method has captured the train quite well. At 30K iterations (~35min) the background artifacts have been reduced significantly. For other scenes (below), the difference is barely visible; 7K iterations (~8min) is already very high quality.

Table 2. PSNR scores for Synthetic NeRF, we start with 100K randomly initialized points. Competing metrics extracted from respective papers.

	Mic	Chair	Ship	Materials	Lego	Drums	Ficus	Hotdog	Avg.
Plenoxels	33.26	33.98	29.62	29.14	34.10	25.35	31.83	36.81	31.76
INGP-Base	36.22	35.00	31.10	29.78	36.39	26.02	33.51	37.40	33.18

- 7K iterations often already produce strong images
- 30K iterations can reduce remaining artifacts
- Trade training time for final quality

Ablation: What Matters Most?

Table 3. PSNR Score for ablation runs. For this experiment, we manually downsampled high-resolution versions of each scene's input images to the established rendering resolution of our other experiments. Doing so reduces random artifacts (e.g., due to JPEG compression in the pre-downscaled Mip-NeRF360 inputs).

	Truck-5K	Garden-5K	Bicycle-5K	Truck-30K	Garden-30K	Bicycle-30K	Average-5K	Average-30K
Limited-BW	14.66	22.07	20.77	13.84	22.88	20.87	19.16	19.19
Random Init	16.75	20.90	19.86	18.02	22.19	21.05	19.17	20.42
No-Split	18.31	23.98	22.21	20.59	26.11	25.02	21.50	23.90
No-SH	22.36	25.22	22.88	24.39	26.59	25.08	23.48	25.35
No-Clone	22.29	25.61	22.15	24.82	27.47	25.46	23.35	25.91
Isotropic	22.40	25.49	22.81	23.89	27.00	24.81	23.56	25.23
Full	22.71	25.82	23.18	24.81	27.70	25.65	23.90	26.05

the difference in visual quality for our two configurations in Fig. 6. be seen in Fig. 10 (second image from the left) and in supplemental

- Removing splitting hurts geometry reconstruction
- Isotropic Gaussians reduce representational flexibility
- Full method achieves the strongest average PSNR

Limitations and Trade-offs

- Large explicit representation increases memory use
- Quality depends on capture coverage and initialization
- Visibility sorting is approximate rather than exact

Key Takeaways

- 3D Gaussians offer differentiable, explicit scene representation
- Adaptive density control improves reconstruction quality
- Tile-based GPU splatting enables real-time novel-view synthesis

Thank You · Q&A

- 3D Gaussian Splatting for Real-Time Radiance Field Rendering
- Presenter 김태하
- Questions?